

Sorokin School



Task-Management System [часть 4]

🔥 Задание к модулю №4 по интенсиву Spring Boot

Сегодня вы поднимаете качество вашего API на новый уровень, добавляя валидацию входных данных и глобальную обработку ошибок. Это позволит вашим клиентам получать понятные и информативные ответы, а вам — легче отлаживать и сопровождать код.

Описание

В задании вам предстоит добавить в проект механизмы валидации данных, используя Spring Bean Validation, и реализовать глобальную обработку ошибок через `@ControllerAdvice`.

Это обеспечит корректную обработку некорректных запросов и информативное сообщение об ошибках в ответах API.

Знания для выполнения (что вам предстоит закрепить)

- Использование Bean Validation (например, `@NotNull`, `@Future` и т.д.) для валидации входящих данных.
- Применение аннотации `@Valid` в контроллерах для автоматической проверки DTO.
- Создание глобального обработчика ошибок с помощью `@ControllerAdvice` и `@ExceptionHandler`.
- Обработка ошибок в бизнес-логике (например, блокировка редактирования задачи в статусе `DONE`).
- Правильное формирование HTTP-ответов с подробной информацией об ошибке

Задание на разработку

🔴 Что конкретно детально нужно реализовать

Валидация сущности Task:

В классе `Task` добавьте Validation аннотации:

- `@NotNull` для поля `creatorId`.
- `@NotNull` и `@Future` для поля `deadlineDate`.
- `@NotNull` для поля `priority`.

Создание класса ошибки: реализуйте следующий класс

```
1 public record ErrorResponseDto(  
2     String message,  
3     String detailedMessage,  
4     LocalDateTime errorTime  
5 ) {}
```

Глобальный обработчик ошибок (GlobalExceptionHandler):

- Создайте класс с аннотацией `@ControllerAdvice` для обработки исключений.
- Реализуйте методы с аннотацией `@ExceptionHandler` для следующих исключений:
 - `Exception`
 - `EntityNotFoundException`
 - `IllegalArgumentException`
 - `IllegalStateException`
 - `MethodArgumentNotValidException`
- Форматируйте ответ в виде JSON с использованием класса `ErrorResponseDto`.

Добавление нового эндпоинта для перевода задачи в DONE:

- В `TaskController` создайте новый метод для обработки запроса:
 - `POST /tasks/{id}/complete`
- В сервисном слое реализуйте логику:
 - Установите новое поле `doneDateTime` (тип `LocalDateTime`) автоматически на текущее время. Это поле добавите в `Task` и `TaskEntity`.
 - Проверьте, что необходимые поля заполнены (например, `assignedUserId` и `deadlineDate`). Если какое-либо из этих полей отсутствует, выбросьте исключение (например, `IllegalArgumentException`) с понятным сообщением.
- При успешном переводе задачи, статус задачи должен измениться на `DONE`.

Советы по реализации

Немного заметок о том как лучше реализовать задание

Убрать обработку ошибок из контроллеров:

Проверьте, чтобы все методы в контроллерах не содержали локальную обработку ошибок — вся обработка должна делегироваться `GlobalExceptionHandler`.

Проверьте бизнес-логику:

В вашем приложении обязательно должна быть реализована следующая бизнес-логика и валидация:

- При создании задачи поле `deadlineDate` должно содержать дату в будущем.
- При переводе задачи в статус `IN_PROGRESS` система проверяет, что поле `assignedUserId` заполнено, и что у данного исполнителя не более 4 активных задач (чтобы в сумме не превышалось 5). Если одно из этих условий не выполняется, должна возвращаться ошибка.
- При редактировании задачи, находящейся в статусе `DONE`, любые изменения блокируются, за исключением ситуации, когда задача переводится из `DONE` обратно в `IN_PROGRESS` для возобновления работы.
- При переводе задачи в статус `DONE` автоматически устанавливается `doneDateTime`, а также проверяется, что обязательные поля (`assignedUserId`, `deadlineDate`) заполнены — в противном случае возвращается ошибка.

Тестирование